

UNIT III

3.0 INTRODUCTION

There are six derived types in C: arrays, functions, pointer, structure, union and enumerated types. The function type is derived from its return type.

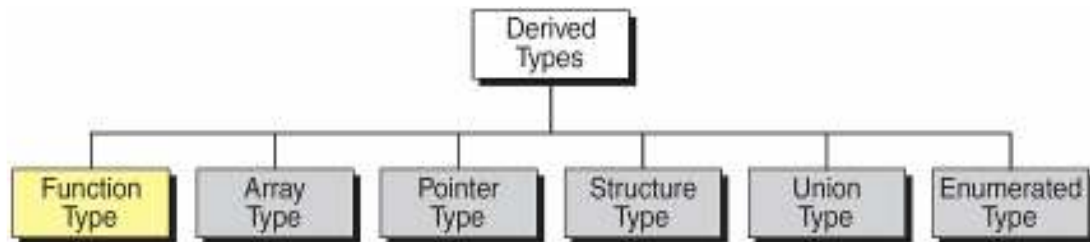


Figure: 3.0 Derived Types

3.1 DESIGNING STRUCTURED PROGRAMS

Whenever we are solving large programs first, we must understand the problem as a whole, then we must break it in to simpler, understandable parts. We call each of these parts of a program a module and the process of sub dividing a problem into manageable parts top-down design.

- ❖ The principles of top-down design and structured programming dictate that a program should be divided into a main module and its related modules.
- ❖ The division of modules proceeds until the module consists only of elementary process that is intrinsically understood and cannot be further subdivided. This process is known as factoring.
- ❖ Top-down design is usually done using a visual representation of the modules known as a structured chart.

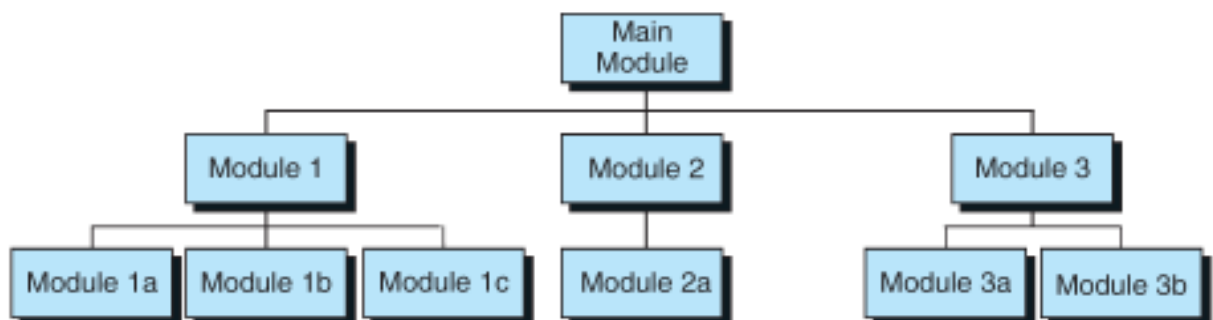


Figure: 3.1 Structure Chart

3.2 FUNCTIONS IN C

A function is a self-contained block of code that carries out some specific and well-defined task.

C functions are classified into two categories

1. Library Functions
2. User Defined Functions

Library Functions

These are the built in functions available in standard library of C. The standard C library is collection various types of functions which perform some standard and predefined tasks.

Example: `abs (a)` function gives the absolute value of `a`, available in `<math.h>` header file

`pow (x, y)` function computes `x` power `y`. available in `<math.h>` header file

`printf ()/scanf ()` performs I/O functions. Etc.,

User Defined Functions

These functions are written by the programmer to perform some specific tasks.

Example: `main ()`, `sum ()`, `fact ()` etc.

The Major distinction between these two categories is that library functions are not required to be written by us whereas a user defined function has to be developed by the user at the time of writing a program.

3.3 USER-DEFINED FUNCTIONS

The basic philosophy of function is divide and conquer by which a complicated tasks are successively divided into simpler and more manageable tasks which can be easily handled. A program can be divided into smaller subprograms that can be developed and tested successfully.

A function is a complete and independent program which is used (or invoked) by the main program or other subprograms. A subprogram receives values called arguments from a calling program, performs calculations and returns the results to the calling program.

3.3.1 ADVANTAGES OF USER-DEFINED FUNCTIONS

1. **Modular Programming** It facilitates top down modular programming. In this programming style, the high level logic of the overall problem is solved first while the details of each lower level functions is addressed later.
2. **Reduction of source code** The length of the source program can be reduced by using functions at appropriate places. This factor is critical with microcomputers where memory space is limited.
3. **Easier Debugging** It is easy to locate and isolate a faulty function for further investigation.
4. **Code Reusability** a program can be used to avoid rewriting the same sequence of code at two or more locations in a program. This is especially useful if the code involved is long or complicated.
5. **Function sharing** Programming teams does a large percentage of programming. If the program is divided into subprograms, each subprogram can be written by one or two team members of the team rather than having the whole team to work on the complex program

3.3.2 THE GENERAL FORM OF A C FUNCTION

```
return_type function_name (argument declaration)
{
    //local declarations
    .....
    .....
    //statements
    .....
    return (expression);
}
```

Figure: 3.2 General Form of A C Function

return-type

Specifies the type of value that a function returns using the return statement. It can be any valid data type. If no data type is specified the function is assumed to return an integer result.

function-name

Must follow same rules of variable names in C. No two functions have the same name in a C program.

argument declaration

Is a comma-separated list of variables that receive the values of the argument when function is called. If there are no argument declaration the bracket consists of keyword void.

A C function name is used three times in a program

1. for function declaration
2. in a function call
3. for function definition.

3.3.3 FUNCTION DECLARATION (OR) PROTOTYPE

The ANSI C standard expands the concept of forward function declaration. This expanded declaration is called a function prototype.

A function prototype performs two special tasks.

1. First it identifies the return type of the function so that the compile can generate the correct code for the return data.
2. Second, it specifies the type and number of arguments used by the function.

The general form of the prototype is

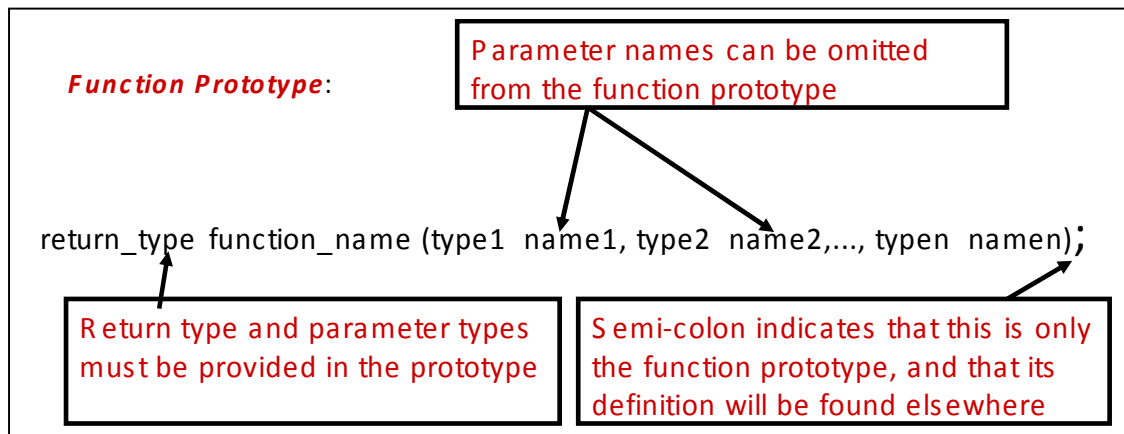


Figure: 3.3 Function Prototype

Note: The prototype normally goes near the top of the program and must appear before any call is made to the function.

3.3.4 THE FUNCTION CALL

A function call is a postfix expression. The operand in a function is the function name. The operator is the parameter lists (...), which contain the actual parameters.

Example:

```
void main ()
{
    sum (a, b);
}
```

When the compiler encounters the function call, the control is transferred to the function `sum()`. The function is executed line by line and outputs the sum of the two numbers and returns to main when the last statement in the following has executed and conceptually, the function's ending `}` is encountered.

3.3.5 FUNCTION DEFINITION

The function definition contains the code for a function. It is made up of two parts: The function header and the function body, the compound statement is must.

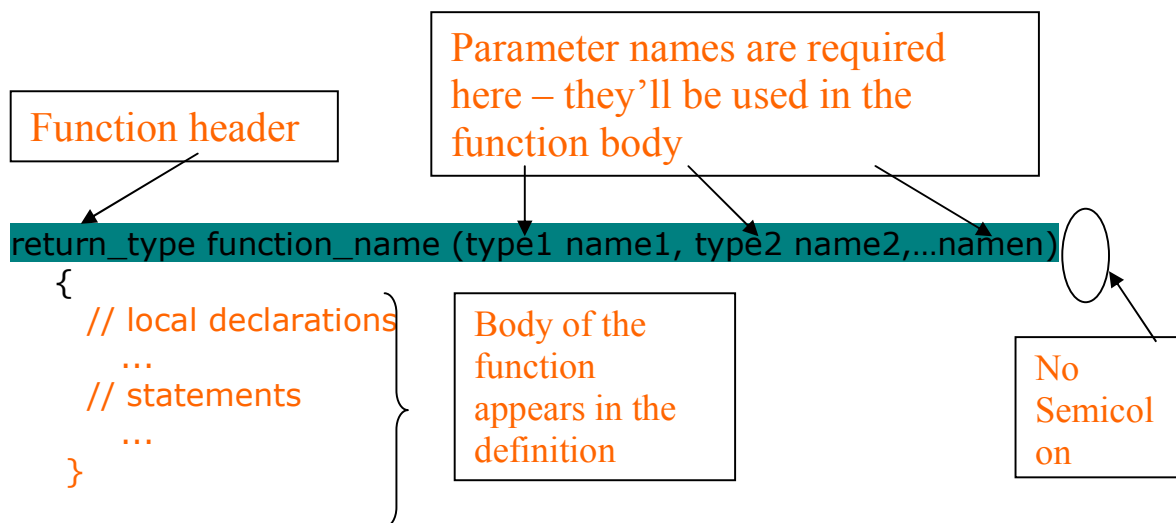


Figure: 3.4 Function Definition

- ❖ Function header consists of three parts: the return type, the function name, and the formal parameter list.
- ❖ function body contains local declarations and function statement.
- ❖ Variables can be declared inside function body.
- ❖ Function can not be defined inside another function.

3.2.6 CATEGORY OF USER -DEFINEDFUNCTIONS

A function, depending on whether arguments are present or not and whether a value is returned or not, may belong to one of the following categories:

- Category 1: Functions with no arguments and no return values
- Category 2: Functions with no arguments and return values
- Category 3: Functions with arguments and no return values
- Category 4: Functions with arguments and return values

3.2.6.1 FUNCTIONS WITH NO ARGUMENTS AND NO RETURN VALUES

This type of function has no arguments, meaning that it does not receive any data from the calling function. Similarly, this type of function will not return any value. Here the calling function does not receive any data from the called function. In effect, there is no data transfer between the calling function and the called function.

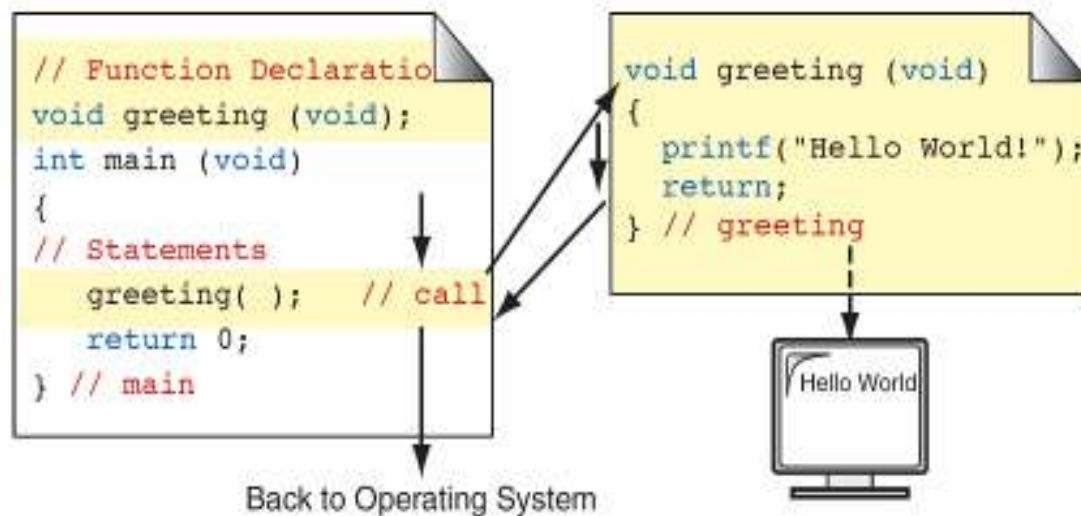


Figure: 3.5 Functions with no arguments and no return values

Observe from the figure 3.5 that the function greeting () do not receive any values from the function main () and it does not return any value to the function main (). Observe the transfer of control between the functions indicated with arrows.

//C program to find sum of two numbers using functions with no arguments and no return values

```

#include<stdio.h>
void sum ();
void main ()
{
    clrscr ();
    sum (); /*calling function */
    getch ();
}
void sum ()
{
    int x, y, z;
    printf ("\n Enter the values of x and y: ");
    scanf ("%d%d", &x, &y);
    z=x+y;
    printf ("\n The sum = %d",z);
}
  
```

3.2.6.2 FUNCTIONS WITH NO ARGUMENTS AND RETURN VALUES

There are two ways that a function terminates execution and returns to the caller.

1. When the last statement in the function has executed and conceptually the function's ending '}' is encountered.
2. Whenever it faces return statement.

THE RETURN STATEMENT

- ❖ The return statement is the mechanism for returning a value from the called function to its caller.
- ❖ The general form of the return statement is

return expression;

- ❖ The calling function is free to ignore the returned value. Further more, there need not be expression after the return.
- ❖ In any case if a function fails to return a value, its value is certain to be garbage.
- ❖ The return statement has two important uses
 1. It causes an immediate exit of the control from the function. That is ,it causes program execution to return to the calling function.
 2. It returns the value present in the expression.

example: return(x+y);
 return (6*8);
 return (3);
 return;

<pre>// Function Declaration int getQuantity (void); int main (void) { // Local Declarations int amt; // Statements amt = getQuantity (); ... return 0; } // main</pre>	<pre>int getQuantity (void) { // Local Declarations int qty; // Statements printf("Enter Quantity"); scanf ("%d", &qty); return qty; } // getQuantity</pre>
--	--

Figure 3.6 Functions with no arguments and return values

In this category, there is no data transfer from the calling function to the called function. But, there is data transfer from called function to the calling function.

In the above example, observe from the figure 3.6 that the function getQuantity () do not receive any value from the function main ().But, it accepts data from the keyboard, and returns the value to the calling function.

// C program to find sum of two numbers using functions with no arguments and return values

```
#include<stdio.h>
int sum ();
void main ()
{
    int c;
    clrscr ();
    c=sum (); /*calling function */
    printf ("\n The sum = %d", c);
    getch ();
}
int sum ()
{
    int x, y;
    printf ("\n Enter the values of x and y: ");
    scanf ("%d%d", &x, &y);
    return x+y;
}
```

RETURNING VALUES FROM MAIN ()

When we use a return statement in main (), some program returns a termination code to the calling process (usually to the O.S).The return values must be an integer.

For many Operating Systems, a return value of '0' indicates that the program terminated normally. All other values indicate that some error occurred. For this reason, it is good practice to use an explicit return statement.

3.2.6.3 FUNCTIONS WITH ARGUMENTS AND NO RETURN VALUES

In this category there is data transfer from the calling function to the called function using parameters. But, there is no data transfer from called function to the calling function.

Local Variables

variables that are defined within a function are called local variables. A local variable comes into existence when the function is entered and is destroyed upon exit.

Function arguments

The arguments that are supplied in to two categories

1. actual arguments/parameters
2. formal arguments/parameters

Actual arguments/parameters

Actual parameters are the expressions in the calling functions. These are the parameters present in the calling statement (function call).

formal arguments/parameters

formal parameters are the variables that are declared in the header of the function definition. These list defines and declares that will contain the data received by the function. These are the value parameters, copies of the values being passed are stored in the called functions memory area.

Note: Actual and Formal parameters must match exactly in type, order, and number. Their names however, do not need to match.

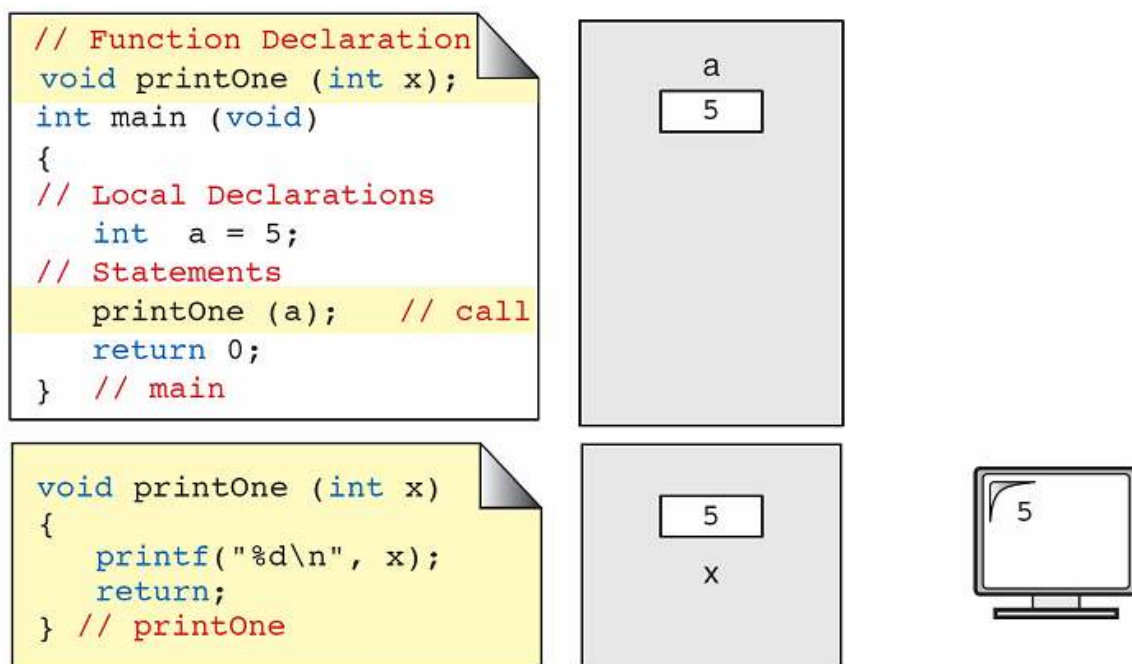


Figure 3.7 Functions with arguments and no return values

Observe from the figure 3.7 that the function `printOne ()` receives one value from the function `main ()`, display the value copy of `a`.

//C program to find sum of two numbers using functions with arguments and no return values

```
#include<stdio.h>
void sum (int ,int );
void main ()
{
    int a, b;
    clrscr ();
    printf ("\n Enter the values of a and b: ");
    scanf ("%d%d", &a, &b);
    sum (a, b); /*calling function */
    getch ();
}
void sum (int x, int y)
{
    int z;
    z=x+y;
    printf ("\n The Sum =%d", z);
}
```

Passing Parameters to Functions

There are two ways of passing parameters to the functions.

1. call by value
2. call by reference

call by value

When a function is called with actual parameters, the values of actual parameters are copied into the formal parameters. If the values of the formal parameters changes in the function, the values of the actual parameters are not changed. This way of passing parameters is called call by value (pass by value).

In the below example, the values of the arguments to swap () 10 and 20 are copied into the parameters x and y. Note that the values of x and y are swapped in the function. But, the values of actual parameters remain same before swap and after swap.

```
// C program illustrates call by value
#include<stdio.h>
void swap (int , int );
void main ()
{
    int a, b;
    clrscr ();
    printf ("\n Enter the values of a and b: ");
    scanf ("%d%d", &a, &b);
    swap (a, b);  /*calling function */
    printf ("\nFrom main The Values of a and b a=%d, b=%d ", a, b);
    getch ();
}
void swap (int x, int y)
{
    int temp;
    temp=x;
    x=y;
    y=temp;
    printf ("\n The Values of a and b after swapping a=%d, b =%d", x, y);
}
OUTPUT
Enter the values of a and b: 10 20
The Values of a and b after swapping a=20, b=10
from main The values of a and b a=10, b=20
```

Note: In call by value any changes done on the formal parameter will not affect the actual parameters.

❖ call by reference will be discussed in UNIT IV.

3.2.6.4. FUNCTIONS WITH ARGUMENTS AND RETURN VALUES

In this category there is data transfer between the calling function and called function.

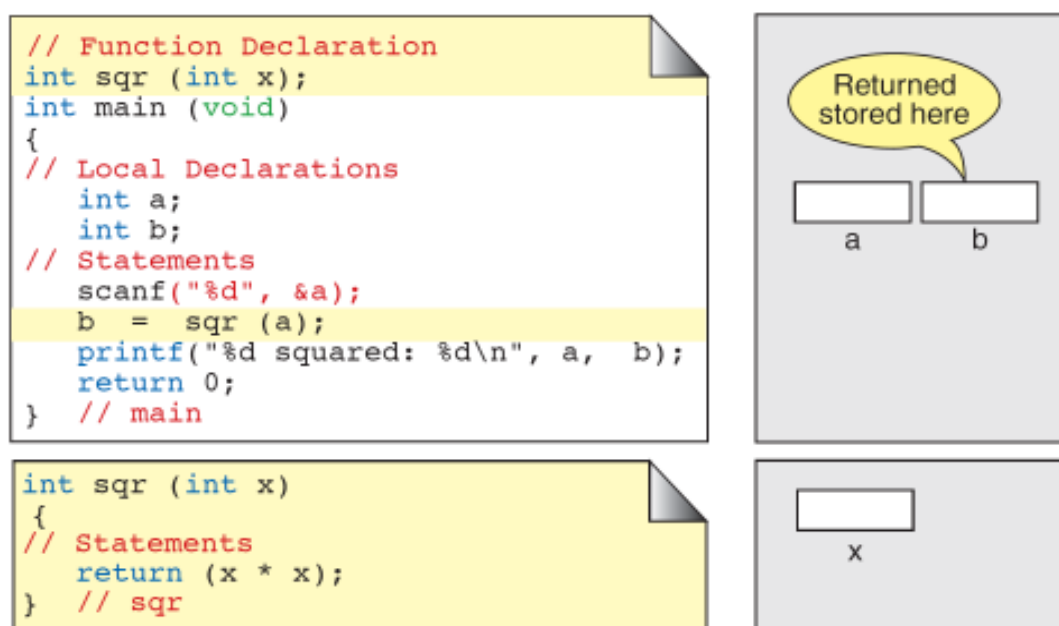


Figure 3.8 Functions with arguments and return values

Observe from the above figure 3.8 that the function `sqr` receives one value from the function `main` (), finds the square of the number and sends the result back to the calling function.

//C program to find sum of two numbers using functions with arguments and return values

```
#include<stdio.h>
int sum (int ,int );

void main ()
{
    int a, b;
    clrscr ();
    printf ("\n Enter the values of a and b: ");
    scanf ("%d%d", &a, &b);
    c=sum (a, b); /*calling function */
    printf ("\n The Sum =%d", c);
    getch ();
}

int sum (int x, int y)
{
    int z;
    return x+y;
}
```

Note: generally we are using functions with arguments and return values (category 4) in our applications. Why because the job of a function is to perform a well-defined task, it carries inputs and sends result after executed. In real world the programming teams codes the modules (functions) according to the input (arguments) given by the team coordinators.

3.3 STANDARD LIBRARY FUNCTIONS

C has a large set of built-in functions that perform various tasks. In order to use these functions their prototype declarations must be included in our program.

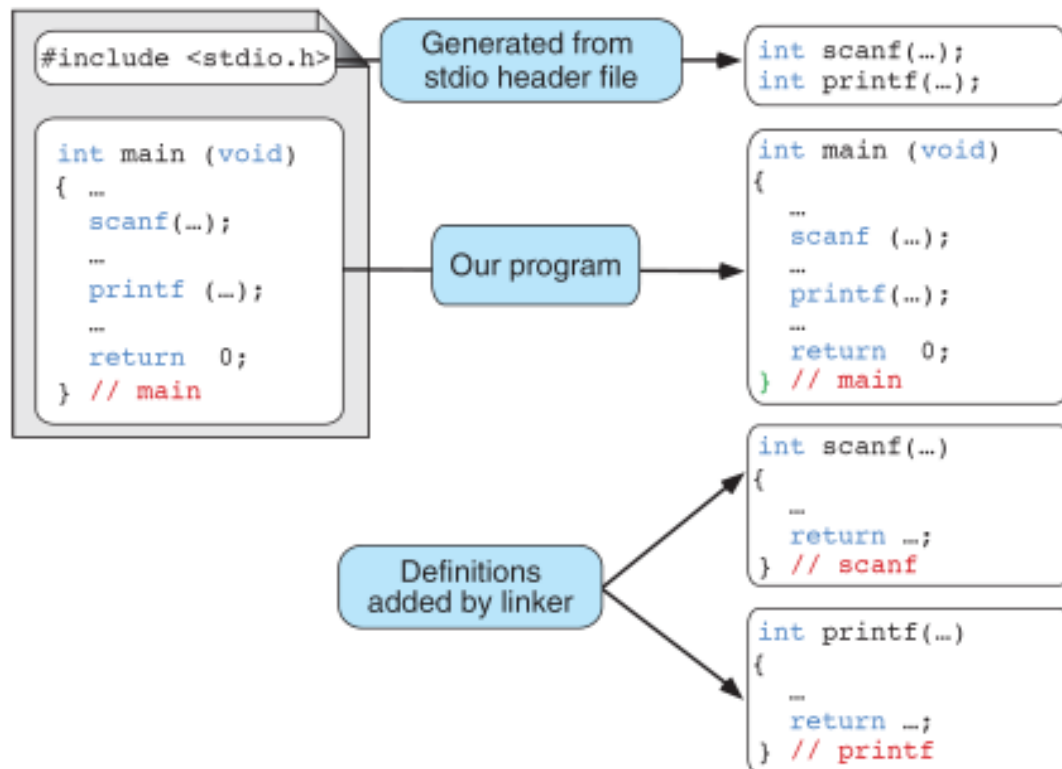


Figure 3.9 Library Functions and the Linker

The above figure shows how two of the C standard functions that we have used several times are brought into our program. The include statement causes the library header file for standard input and output (`stdio.h`) to be copied into our program. It contains the declaration for `printf` and `scanf`. Then, when the program is linked, the object code for these functions is combined with our code to build the complete program.

Some of the header files that include these functions are

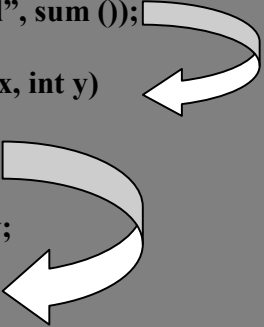
- <stdio.h> Standard I/O functions
- <stdlib.h> Utility functions such as string conversion routines, memory allocation routines, etc.,
- <string.h> String manipulation functions
- <math.h> Mathematical functions
- <ctype.h> Character testing and conversion functions

3.4 NESTING OF FUNCTIONS

C permits nesting of functions, main can call function1, which calls function2, which calls function3 .., There is no limit as how deeply functions can be b=nested .

consider the following example:

```
#include<stdio.h>
int read ();
int sum (int, in t);
void main ()
{
    int a, b;
    printf ("%d", sum ());
}
int sum (int x, int y)
{
    x=read ();
    y=read ();
    return x+y;
}
int read ()
{
    int p;
    printf ("\n Enter any value: ");
    Scanf ("%d", &p);
    return p;
}
```



In the above example ,when the main() function executes it finds the function call sum(), then the control is transferred from main() to the function sum(),here we are calling the function read(), then the control transferred to read() function,then the body of the function read() executes,the control transfered from read to sum() and once again the same is done for rading some other value. Then the addition is performed this value is carried from sum() to main().Observe the chain of control transfers between the nested functions.

3.5 RECURSION IN C

In C, functions can call themselves. A function is recursive if a statement of the function calls the function that contains it. This process is sometimes called circular definition.

Recursion is a repetitive process, where the function calls itself.

Concept of recursive function:

- ❖ A recursive function is called to solve a problem
- ❖ The function only knows how to solve the simplest case of the problem. When the simplest case is given as an input, the function will immediately return with an answer.
- ❖ However, if a more complex input is given, a recursive function will divide the problem into 2 pieces: a part that it knows how to solve and another part that it does not know how to solve. The part that it does not know how to solve resembles the original problem, but of a slightly simpler version.
- ❖ Therefore, the function calls itself to solve this simpler piece of problem that it does now know how to solve. This is what called the recursion step.
- ❖ The statement that solves problem is known as the base case. Every recursive function must have a base case. The rest of the function is known as the general case.
- ❖ The recursion step is done until the problem converges to become the simplest case.
- ❖ This simplest case will be solved by the function which will then return the answer to the previous copy of the function.
- ❖ The sequence of returns will then go all the way up until the original call of the function finally return the result.

Example: Recursive factorial function

Iteration definition

$$\begin{array}{ll} \text{fact}(n) = 1 & \text{if } n=0 \\ = n*(n-1)*(n-2)\dots\dots 3*2*1 & \text{if } n>0 \end{array}$$

Recursion definition

$$\begin{array}{ll} \text{fact}(n) = 1 & \text{if } n=0 \quad \textbf{BASE CASE} \\ = n*\text{fact}(n-1) & \text{if } n>0 \quad \textbf{GENERAL CASE} \end{array}$$

//C program to find the factorial using recursion

```

#include <stdio.h>
long factorial (long);
void main (void)
{
    int i;
    i=4;
    printf ("%2d! = %1d\n", i, factorial (i));
}
long factorial (long number)
{
    if (number ==0)
        return 1;
    else
        return (number * factorial (number-1));
}

```

• Output:

4! = 24

designing a recursive function:

In the above program, once the base condition has reached, the solution begins. The program has found one part of the answer and can return that part to the next more general statement. Thus, after calculating factorial (0) is 1, and then it returns 1. That leads to solve the next general case,

$$\text{factorial (1)} \rightarrow 1 * \text{factorial (0)} \rightarrow 1 * 1 \rightarrow 1$$

The program now returns the value of factorial (1) to next general case, factorial (2),

$$\text{factorial (2)} \rightarrow 2 * \text{factorial (1)} \rightarrow 2 * 1 \rightarrow 2$$

As the program solves each general case in turn, the program can solve the next higher general case, until it finally solves the most general case, the original problem.

The following are the rules for designing a recursive function:

1. First, determine the base case.
2. Then, determine the general case.
3. Finally, combine the base case and general case in to a function.

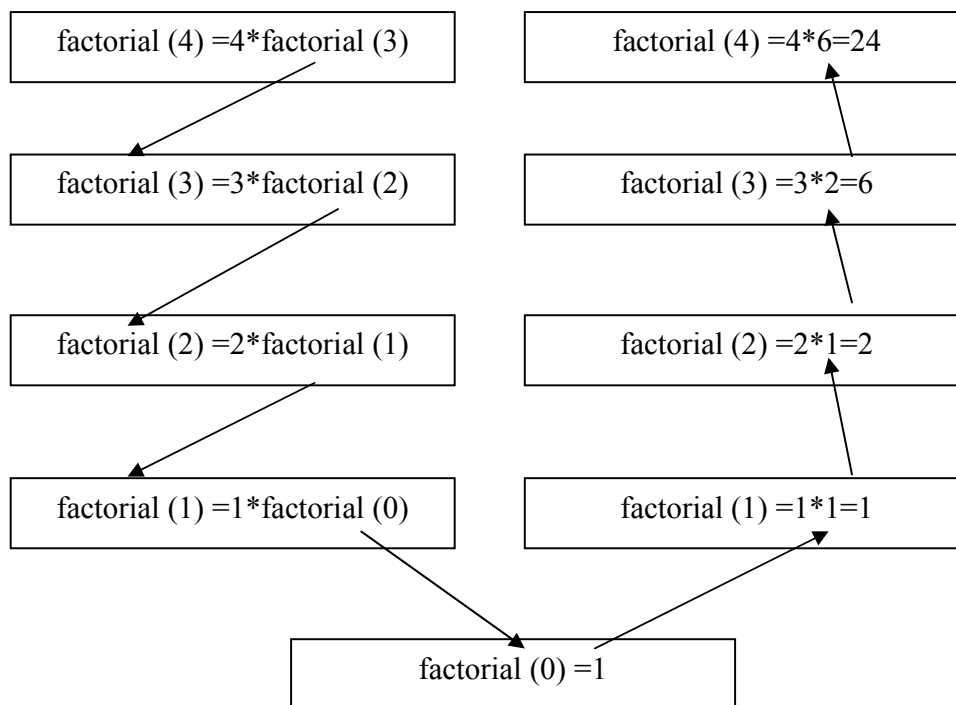


Figure 3.10 factorial (4) recursively

Difference between Iteration and Recursion

ITERATION	RECURSION
<i>Iteration explicitly uses repetition structure.</i>	Recursion achieves repetition by calling the same function repeatedly.
<i>Iteration is terminated when the loop condition fails</i>	Recursion is terminated when base case is satisfied.
<i>May have infinite loop if the loop condition never fails</i>	Recursion is infinite if there is no base case or if base case never reaches.
<i>Iterative functions execute much faster and occupy less memory space.</i>	Recursive functions are slow and takes a lot of memory space compared to iterative functions

Table 3.1 Difference between Iteration and Recursion

3.6 PREPROCESSOR COMMANDS

The C compiler is made of two functional parts: a preprocessor and a translator. The preprocessor is a program which processes the source code before it passes through the compiler. The translator converts the C statements into machine code that in places in an object module.

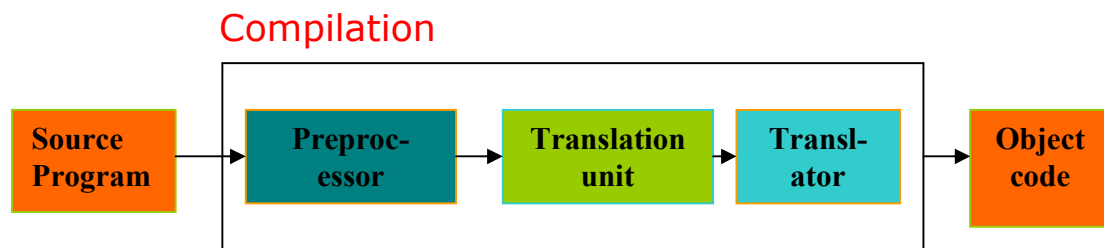


Figure 3.11 Compilation Components

The preprocessor is a collection of special statements called commands or preprocessor directives. Each directive is examined by the preprocessor before the source program passes through the compiler.

- ❖ Statements beginning with # are considered preprocessor directives.
- ❖ Preprocessor directives indicate certain things to be done to the program code prior to compilation.
- ❖ It includes certain other files when compiling, replace some code by other code.
- ❖ Only white space characters before directives on a line.
- ❖ Preprocessor commands can be placed anywhere in the program.

There are three major tasks of a preprocessor directive

1. Inclusion of other files (file inclusion)
2. Definition of symbolic constants and macros(macro definition)
3. Conditional compilation of program code/Conditional execution of preprocessor directives

3.6.1 FILE INCLUSION

The first job of a preprocessor is file inclusion ,the copying of one or more files into programs. The files are usually header files and external files containing functions and data declarations.

General form is,

#include filename

it has two different forms

1. #include <filename>

- ❖ it is used to direct the preprocessor to include header files from the system library.
- ❖ in this format , the name of the header file is enclosed in **pointed brackets**.
- ❖ Searches standard library only for inclusion of a file.

example

#include<stdio.h>

In the above example ,the preprocessor directive statements searches for content of entire code of the header file stdio.h in the standard library ,if it finds there includes the contents of the entire code at the place where the statement is in the source program before the source program passes through the compiler.

Note : here we can't include user files why because it searches for the files in the standard library only.

2. #include "filename"

- ❖ it is used to direct the preprocessor look for the files in the current working directory, standard library
- ❖ Use for inclusion of user-defined files, also includes standard library files.
- ❖ Searches current directory, then standard library for the inclusion of a file.
- ❖ in this format , the name of the file is enclosed in **double quotes**.

- ❖ we can also include macros. The included file or the program either may contain main() function but not the both.

Example

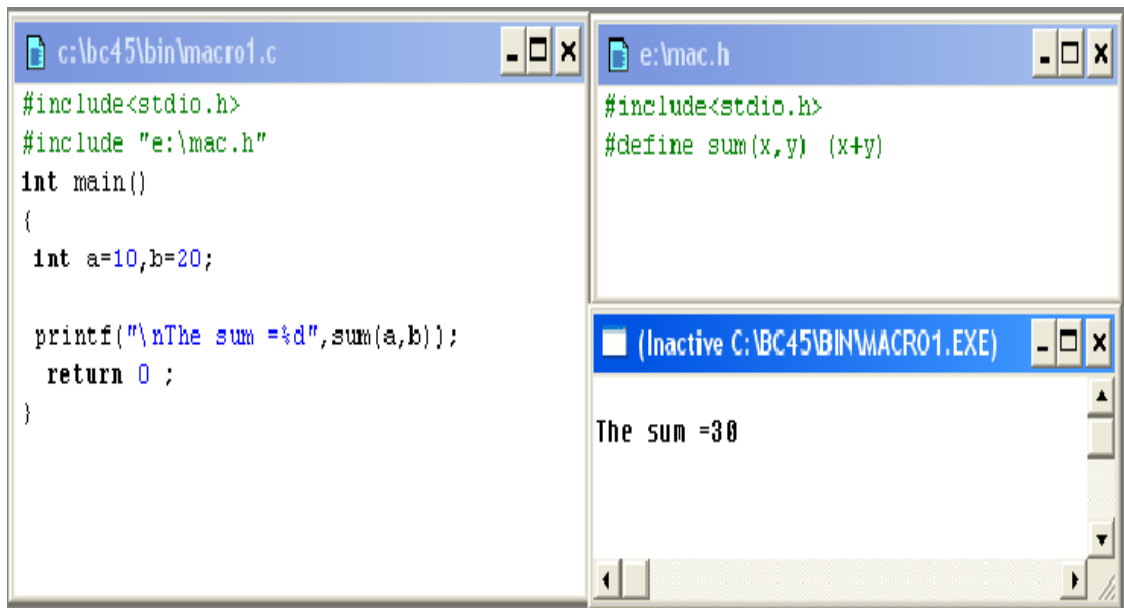
#include "header.h"

In the above example ,the preprocessor directive statement directs the preprocessor to include content of entire code of the file header.h which is in the current directory at the place where the statement is in the source program before the source program passes through the compiler.

note : We can also include file that is not present in the current directory by specifying the complete path of the file like

#include "e:\muc.h"

//C Program illustrates file inclusion



```

c:\bc45\bin\macro1.c
#include<stdio.h>
#include "e:\mac.h"
int main()
{
    int a=10,b=20;

    printf("\nThe sum =%d",sum(a,b));
    return 0 ;
}

e:\mac.h
#include<stdio.h>
#define sum(x,y) (x+y)

(Inactive C:\BC45\BIN\MACRO1.EXE)
The sum =30
  
```

In the above example the file **mac.h** is stored in **E:** Drive of the computer. After finding the include command by the preprocessor the contents of the file are copied in to the program then send for the compilation.

3.6.2 MACRO DEFINITION

A macro definition command associates a name with a sequence of tokens. The name is called the macro name and the tokens are referred to as the macro body.

The following syntax is used to define a macro:

#define macro_name(<arg_list>) macro_body

#define is a define directive, macro_name is a valid C identifier. macro_body is used to specify how the name is replaced in the program before it is compiled.

❖ Example

#define CIRCLE_AREA(x) (PI * (x) * (x))

would cause

area = CIRCLE_AREA(4);

to become

area = (3.14159 * (4) * (4));

❖ Use parenthesis for coding macro_body, Without them the macro

#define CIRCLE_AREA(x) PI * (x) * (x)

would cause

area = CIRCLE_AREA(c + 2);

to become

area = 3.14159 * c + 2 * c + 2;

❖ We can also supply Multiple arguments like

#define RECTANGLE_AREA(x, y) ((x) * (y))

would cause

```
rectArea = RECTANGLE_AREA( a + 4, b + 7 );
```

to become

```
rectArea = ( ( a + 4 ) * ( b + 7 ) );
```

- ❖ macro must be coded on a single line. we can use backslash(\) followed immediately by a new line to code macro in multiple lines.
- ❖ Performs a text substitution – no data type checking. We need to carefully code the macro body. Whenever a macro call is encountered, the preprocessor replaces the call with the macro body. If body is not created carefully it may create an error or undesired result.

For example the macro definition

```
#define Mac =0
.....
.....
a=Mac;
....
will result in
    num==0;
```

This is unwanted.

//C program Illustrating usage of Macro

```
#include<stdio.h>
#define square(x) (x*x)
int main()
{
    int a=10;

    printf("\nThe square of %d=%d",a,square(a));
    return 0 ;
}
```

OUTPUT

The square of 10=100

Symbolic Constants

Macro definition without arguments is referred as a constant. The body of the macro definition can be any constant value including integer, float, double, character, or string. However, character constants must be enclosed in single quotes and string constants in double quotes.

Example

```
#define PI 3.14159
```

Here "PI" replaces with "3.14159"

Nested Macros

It is possible to nest macros. C handles nested macros by simply rescanning a line after macro expansion. Therefore, if an expansion results in a new statement with a macro, the second macro will be properly expanded.

For example,

```
#define sqre(a) (a*a)  
#define cube(a) (sqre(a)*a)
```

The expansion of

```
x=cube(4);
```

results in the following expansion:

```
x=(sqre(4)*4);
```

this after rescanning becomes

```
x=((4*4)*4);
```

Undefining Macros

Once defined, a macro command cannot be redefined. Any attempt to redefine it will result in a compilation error. However, it is possible to redefine a macro by first undefining it, using the `#undef` command and then defining it again.

Example

```
#define Val 30
```

```
...
```

```
#undef Val
```

```
#define val 50
```

Predefined Macros

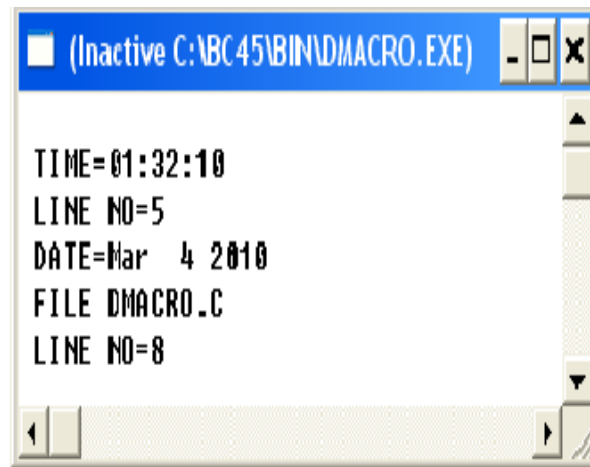
C language provides several predefined macros. These macros cannot be undefined using the `undef` command. Some of the predefined macros are

Command	Meaning
__DATE__	Provides a string constant in the form "mm dd yyyy" containing the date of translation.
__FILE__	Provides a string constant containing the name of the source file.
__TIME__	Provides a string constant in the form "hh:mm:ss" containing the time of the translation.
__LINE__	Provides an integer constant containing the current statement number in the source file.

Table 3.2 List of Predefined Macros in C

//C program Demonstrating Predefined macros

```
#include<stdio.h>
int main()
{
    printf("\n TIME=%s",__TIME__);
    printf("\n LINE NO=%d",__LINE__);
    printf("\n DATE=%s",__DATE__);
    printf("\n FILE %s",__FILE__);
    printf("\n LINE NO=%d",__LINE__);
    return 0;
}
```

**String Converting Operator(#)**

It is a macro operation Causes a replacement text token to be converted to a string surrounded by quotes.

The statement

```
#define HELLO( x ) printf( "Hello, " #x "\n" );
```

would cause

```
HELLO( John )
```

to become

```
printf( "Hello, " "John" "\n" );
```

Strings separated by whitespace are concatenated when using printf statement.

The Defined Operator

The defined operator can be used only in a conditional compilation . It can be used in macros. The value of defined(macro-name) is 0 if the name is not defined and 1 if it is defined.

For example, after

#define val 24

the value of **defined(val)** is 1 and

the value of **!defined(val)** is 0.

3.6.3 CONDITIONAL COMPILATION

- ❖ It allows us to control the compilation process by including or excluding statements.
- ❖ Cast expressions, size of, enumeration constants cannot be evaluated in preprocessor directives.
- ❖ Its structure similar to if statement.

❖ Syntax

```
#if expression1
    code to be included for true
#elif expression2
    code to be included for true
#else
    code to be included false
#endif
```

❖ Example,

```
#if !defined( NULL )
    #define NULL 0
#endif
```

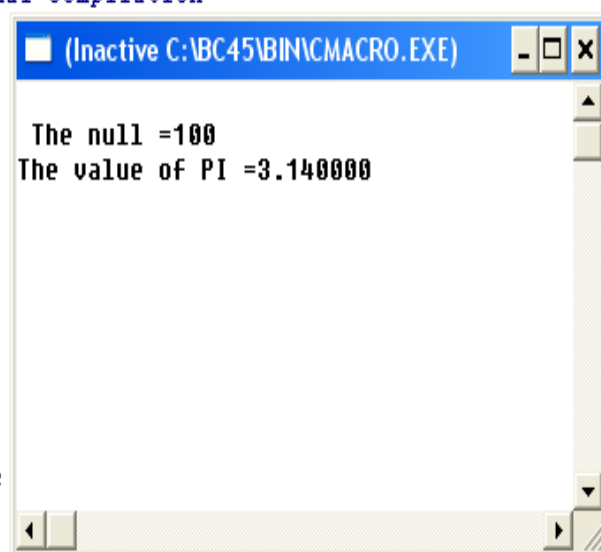
- ❖ Determines if symbolic constant NULL has been defined If NULL is defined, defined(NULL) evaluates to 1, If NULL is not defined, and this function defines NULL to be 0.
- ❖ Every #if must end with #endif.
- ❖ #ifdef short for #if defined(name).
- ❖ #ifndef short for #if !defined(name).

Command	Meaning
#if expression	When expression is true, the code that follows is included for compilation.
#endif	Terminates the conditional command.
#else	Specifies alternate code in two-way decision.
#elif	Specifies alternate code in multi-way decision.
#ifdef name	Tests for the macro definition.
#ifndef name	Tests whether macro is not defined .

Table 3.3 Conditional Compilation Commands

// C program demonstrating Conditional Compilation

```
#include<stdio.h>
#define val 1
int main()
{
    #if !defined(null)
        #define null 100
    #endif
    #ifdef val
        #define pi 3.14
    #endif
    printf("\n The null =%d",null);
    printf("\nThe value of PI =%f",pi);
    return 0;
}
```



3.7 SCOPE

Scope determines the region of the program in which a defined object is visible. That is, the part of the program in which we can use the object's name. Scope pertains to any object that can be declared, such as a variable or a function declaration.

In C, an object can have three levels of scope

1. Block scope
2. File scope
3. Function scope

Scope Rules for Blocks

- ❖ Block is zero or more statements enclosed in a set of braces.
- ❖ Variables are in scope from their point of declaration until the end of the block. Block scope also referred as local scope.
- ❖ Variables defined within a block have a local scope. They are visible in that block scope only. Outside the block they are not visible.
- ❖ For example, a variable declared in the formal parameter list of a function has block scope, and active only in the body only. Variable declared in the initialization section of a for loop has also block scope, but only within the for statement.

```

{
    int a = 2;                /* outer block a */
    printf ("%d\n", a);      /* 2 is printed */
    {
        int a = 5;          /* inner block a */
        printf ("%d\n", a); /* 5 is printed */
    }
    printf ("%d\n", a);      /* 2 is printed */
}

/* a no longer defined */

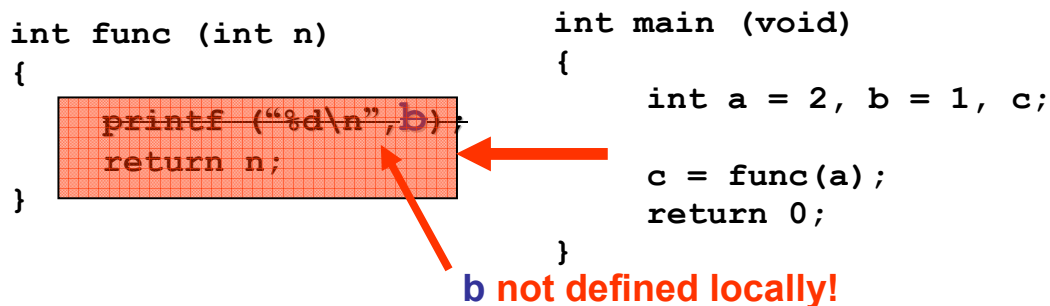
```

Outer" a "Masked

- ❖ In the above example the variable `a` is created with value 2 and its scope limit to the block and it is local to that block. Here we are creating the same identifier with another value 5 in the inner block, 5 is displayed on the screen after executing inner `printf` statement. The `a` value 2 is once again in the outer block. After the block `a` is not visible.
- ❖ A variable that is declared in an outer block is available in the inner block unless it is redeclared. In this case the outer block declaration is temporarily "masked".
- ❖ Avoid masking! Use different identifiers instead to keep your code debuggable!

Scope rules for functions

- ❖ Variables defined within a function (including `main`) are local to this function and no other function has direct access to them!
- ❖ The only way to pass variables to a function is as parameters.
- ❖ The only way to pass (a single) variable back to the calling function is via the **return** statement.
- ❖ In the function scope global variable and pointer variable are exceptions, and visible here.



```

int func (int n)
{
    printf ("%d\n", b);
    return n;
}

int main (void)
{
    int a = 2, b = 1, c;
    c = func(a);
    return 0;
}

```

b not defined locally!

Figure 3.12 Example for function scope

Scope rules for files

- ❖ File scope includes the entire source file for a program, including any files included in it.
- ❖ An object with file scope has visibility through the whole source file in which it is declared.
- ❖ Objects within block scope are excluded from file scope unless specifically declared to have file scope; in other words, by default block scope hides objects from file scope.

- ❖ File scope generally includes all declarations outside function and all function headers.
- ❖ An object with file scope sometimes referred to as a global object.
- ❖ For Example, a variable declared in the global section can be accessed from anywhere in the source program.

Local Variables

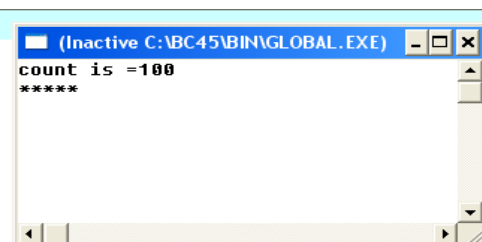
- ❖ Variables that are declared in a block known as local variables. These variables may be any variable in a function also.
- ❖ They are known in the block where they are created. Active in the block only.
- ❖ They hold their values during the execution of the block. After completing the execution of the block they are undefined.

Global Variables

- ❖ Global variables are known throughout the entire program and may be used in any piece of code.
- ❖ Also, they will hold their values during the entire execution of the program.
- ❖ Global variables are created by declaring them outside of the function and they can be accessed by any expression.

//C Program Illustrates Global Variable

```
#include<stdio.h>
void fun1();
void fun2();
int count;
int main()
{
    count=100;
    fun1();
    getch();
    return 0;
}
void fun1()
{
    printf("count is =%d\n",count);
    fun2();
}
void fun2()
{
    int count;
    for(count =0;count<5;count++)
        putchar('*');
}
```



- ❖ In the above program, you can see that the variable **count** has been declared outside of all functions. Its declaration comes before the **main** function. However, it could have been placed anywhere prior to its first use, as long as it was not in a function.
- ❖ In general we are declaring global variable at the top of the program.
- ❖ Look at the program fragment, it should be clear that although neither **main ()** nor **fun1 ()** have declared the variable **count**, both may use it.
- ❖ However, **fun2 ()** has declared a local variable **count**. When **fun2 ()** references **count**, it will be referencing only its local variable, not the global one.
- ❖ Remember that if a global variable and a local variable have the same name, all references to that name inside the function where the local variable is declared refer to the local variable and have no effect on the global variable. This is a convenient benefit. However, failing to remember this can cause your program to act strangely, although it looks correct.
- ❖ Storage for global variable is in fixed memory region of memory set aside for this purpose by the compiler.
- ❖ Global variable are very helpful when the same data is used in the many functions in your computer program.

- ❖ **you should avoid using unnecessary global variables, for four reasons**
 1. They take up the entire time and memory of your program while execution and not just when they are needed.
 2. Avoid using global variables to pass parameters to functions, only when all variables in a function are local, it can be used in different programs, Global variables are confusing in long code.
 3. Using a large number of global variables can lead to program errors because of unknown and unwanted name clashes.
 4. Global variables compromise the security of the program.

3.8. STORAGE CLASSES

Each and every variable is storing in the computer memory. Every variable and function in C has two attributes:

type (**int,float,...**)
storage class

Storage class specifies the scope of the object. It also provides information about location and visibility of the variable.

Object Storage Attributes

The storage class specifies control three attributes of an object's storage as shown in below figure 3.13. Its, scope, extent, and linkage.

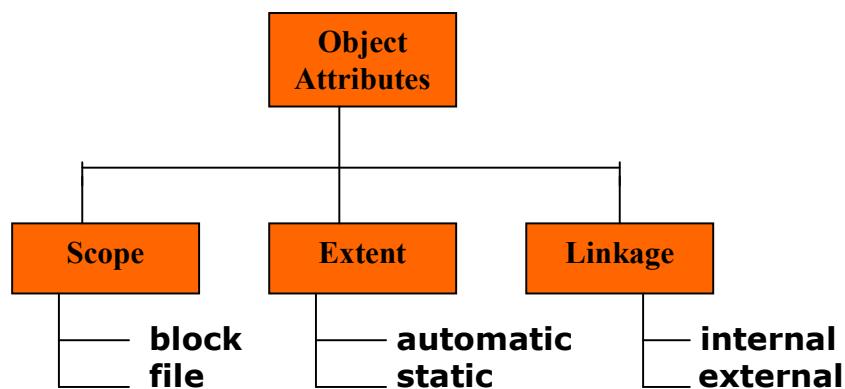


Figure 3.13 Object Storage Attributes

Scope

We have discussed this one in the previous section 3.7.

Extent

The **extent** of an object defines the duration for which the computer allocates memory for it, also known as life-time of an object. In C, an extent can have automatic, static extent, or dynamic extent.

Automatic Extent

An object with an automatic extent is created each time its declaration is encountered and is destroyed each time its block is exited.

For Example, a variable in the body of a loop is created and destroyed in each iteration.

Static Extent

A variable with a static extent is created when the program is loaded for execution and is destroyed when the execution stops. This is true no matter how many times the declaration is encountered during the execution.

Dynamic Extent

Dynamic extent is created by the program through malloc and its related functions.

Linkage

A large application program may be broken into several modules, with each module potentially written by a different programmer. Each module is a separate source file with its own objects.

We can define two types of **linkages**: internal and external

Internal Linkage

An object with an internal linkage is declared and visible in one module. Other modules cannot refer to this object.

External Linkage

An object with an external linkage is declared in one module but is visible in all other modules that declare it with a special keyword, extern.

Storage Class Specifiers

There are four storage classes

- ❖ auto
- ❖ extern
- ❖ register
- ❖ static

Auto Variables

- ❖ A variable with an **auto** specification has the following storage characteristic:

Scope: block Extent: automatic Linkage: internal

- ❖ The default and the most commonly used storage class is auto.
- ❖ Memory for automatic variables is allocated when a block or function is entered. They are defined and are "local" to the block.
- ❖ When the block is exited, the system releases the memory that was allocated to the auto variables, and their values are lost.
- ❖ These are also referred with local variables

Declaration:

auto type variable_name;

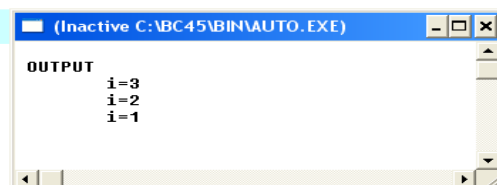
There's no point in using auto, as it's implicitly there anyway. By default all variables are created as automatic variables.

Initialization

An auto variable can be initialized where it is defined or left uninitialized. When auto variables are not initialized its value is garbage value.

// C Program to illustrate the auto storage class

```
#include<stdio.h>
int main()
{
    auto int i=1;
    {
        auto int i=2;
        {
            auto int i=3;
            printf("\n OUTPUT \n");
            printf("\ti=%d",i);
        }
        printf("\n\ti=%d",i);
    }
    printf("\n\ti=%d",i);
    getch();
}
```



In the above example, the compiler treats the three i's as total different variables, since they are defined in different blocks. Once the control comes out of the innermost block the variable i with value 3 is lost, and hence, i in the second printf () refers to i with value 2. Similarly, when the control comes out of the next innermost block, the third printf () refers to i with value 1.

Register Variable

- ❖ register tells the compiler that the variables should be stored in high-speed memory registers if possible. This is done for efficiency.
- ❖ Only a few such registers are available, and can be assigned to frequently-accessed variables if execution time is a problem.
- ❖ If no register space is free, variables become auto instead. Keep registers in small local blocks which are reallocated quickly.
- ❖ A register variable address is not available to the user .this means we can't use the address operator and the indirection operator (pointer) with a register.
- ❖ A variable with **register** specification has the following storage characteristic:

Scope: block Extent: automatic/Register Linkage: internal

Declaration

register type variable_Name;

Initialization

A register variable can be initialized where it is defined or left uninitialized. When auto variables are not initialized its value is garbage value.

Example,

```
#include<stdio.h>
int main ()
{
    register double i=1;
    for (i=1; i<=1000; i++)
    {
        printf (" i=%d", i);
    }
}
```

 register is freed on block exit

Static Variables

The static specifier has different usages depending on its scope.

1. Static Variables with Block Scope

- ❖ A static variable that is declared in a block scope, **static** defines the extent of the variable.
- ❖ The computer allocates storage only once. The linkage is internal, and not visible to other blocks.
- ❖ Also Known as static Local Variable.
- ❖ A variable with an **static block** specification has the following storage characteristic:

Scope: block Extent: static Linkage: internal

Declaration

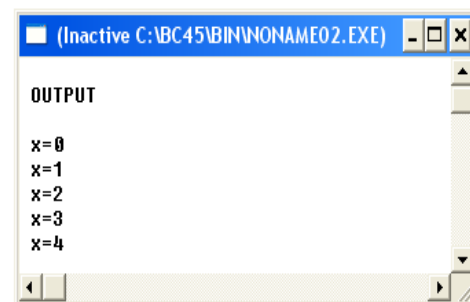
static type variable_Name;

Initialization

A static variable name can be initialized where it is defined, or it can be left uninitialized. If it is initialized, it is initialized only once. If not initialized, its value will be initialized to zero.

// C Program illustrating static storage class in block

```
#include<stdio.h>
void fun1();
int main()
{
    int i=1;
    printf("\n OUTPUT \n");
    for(i=1;i<=5;i++)
        fun1();
    getch();
    return 0;
}
void fun1()
{
    static int x=0;
    printf("\n x=%d",x++);
}
```



Note that in the above program variable *i* is an auto variable. The variable *x*, however, is a static variable. It is only initialized once although the declaration is encountered 5 times. If we do not initialize *x* to 0 because a static value needs an initialization value.

Note: The difference between a static local variable and a global variable is that the static local variable remains known only to the block in which it is declared. Global variable is visible to all the blocks in the program.

2. Static Variable with File Scope

- ❖ When the static specifier is used with a variable that has file (global) scope and we want to keep its internal linkage, it is defined with specifier **static**.
- ❖ A variable with an **static file** specification has the following storage characteristic:

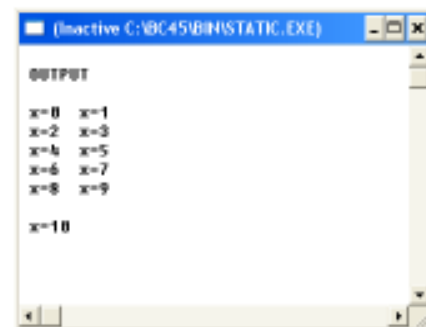
Scope: file

Extent: static

Linkage: internal

//C Program illustrating static specifier with file scope

```
#include<stdio.h>
void fun1();
static int x;
int main()
{
    int i=0;
    printf("\n OUTPUT \n\n");
    for(i=1;i<=5;i++)
    {
        printf(" x=%d ",x++);
        fun1();
    }
    printf("\n x=%d",x);
    return 0;
}
void fun1()
{
    printf(" x=%d\n",x++);
}
```



The declaration of the file scope must be in the global area of the file. If there is another declarations with the same identifier in the global area, we get an error message. In the above program x is declared in global area, it is visible for all blocks in the program. Here we can observe the control transfer between the function block.

External Variables

- ❖ External variables are used with separate compilations. It is common, on large projects, to decompose the project into many source files.
- ❖ The decomposed source files are compiled separately and linked together to form one unit.
- ❖ Within file variables outside functions have external storage class, even without the keyword **extern**.
- ❖ Global variables (defined outside functions) and all functions are of the storage class **extern** and storage is permanently assigned to them.
- ❖ Files can be compiled separately, even for one program. **extern** is used for global variables that are shared across code in several files.

- ❖ a variable declared with a storage class of extent has a file scope; the extent is, static, but linkage is external.

Scope: file Extent: static Linkage: external

Declaration

extern type variable_Name;

An external variable before must be declared before that is used by other source files. The above syntax is used to external variables from other files.

Initialization

If the variable not initialized, then the first declaration seem by the linkage is considered the definition and all other declarations reference it. If an initializer is used with a global definition it is defining declaration.

// extern in multi-file projects

file1.c

```
#include <stdio.h>
int a =1, b = 2, c = 3;  /* external variables */
int f (void);
int main (void)
{
    printf ("%3d\n", f( ));
    printf ("%3d%3d%3d\n", a, b, c); — print 4, 2, 3
    return 0;
}
```

a is global and changed by f

file2.c

```
int f (void)
{
    extern int    a;  /* look for it elsewhere */
    int          b, c;
    a = b = c = 4; — b and c are local and don't survive
    return (a + b + c);
}
```

return 12

Note: The difference between a normal static variable and a extern static variable is that the static external variable is available only within the file where it is defined while the simple external variable can be accessed by other files.

Class	Scope	Extent	Linkage	Initial Value
auto	block	automatic	internal	indeterminate
register	block	automatic	internal	indeterminate
static(extern)	block	static	internal	Zero
static(linkage)	file	static	internal	Zero
extern	file	static	external	indeterminate

Table 3.4 Summary of Storage Classes

3.9 ARRAYS IN C

3.9.1 INTRODUCTION

- ❖ Often we need many different variables that are of the same type and play a similar role in the program
- ❖ For example, suppose we have a temperature reading for each day of the year and need to manipulate these values several times within a program.
- ❖ With simple variables,
 - We would need to declare 365 variables.
 - We would not be able to loop over these variables.
- ❖ An array is a numbered collection of variables of the same type.
- ❖ An array is a homogeneous collection of data.
- ❖ The variables in an array share the same name and are distinguished from one another by their numbers or subscripts.
- ❖ We can loop through the variables of an array by using a variable for the subscript.
- ❖ The subscripts are always 0,1,..., size-1

3.9.2 ARRAY CHARACTERISTICS

- ❖ An array represents a group of related data.

- ❖ An array has two distinct characteristics:

An array is ordered: data is grouped sequentially. In other words, here is element 0, element 1, etc.

An array is homogenous: every value within the array must share the same data type. In other words, an int array can only hold ints, not doubles.

3.9.3 HOW TO CREATE AN ARRAY

- ❖ Before we create an array, we need to decide on two properties:

Element type: what kind of data will your array hold? ints, double, chars? Remember, we can only choose one type.

Array size: how many elements will your array contain?

When we initially write our program, we must pick an array size. An array will stay this size throughout the execution of the program. In other words, we can change the size of an array at compile time, but cannot change it at run-time.

3.9.3 USING ARRAYS IN C

In C, the arrays can be classified based on how the data items are arranged for human understanding

Arrays are broadly classified into three categories,

1. One dimensional arrays
2. Two dimensional arrays
3. Multi dimensional arrays

3.9.4 ONE DIMENSIONAL ARRAYS

One dimensional array is a linear list consisting of related and similar data items. In memory all the data items are stored in contiguous memory locations one after the other.

3.9.4.1 DECLARING ONE DIMENSIONAL ARRAYS

- ❖ To declare regular variables we just specify a data type and a unique name:

int number;

- ❖ To declare an array, we just add an array size.
- ❖ For example:

int temp[5];

Creates an array of 5 integer elements.

- ❖ For example:

double stockprice[31];

creates an array of 31 doubles.

Syntax for Declaring one dimensional Arrays

elementType arrayName [size];

Where :

elementType: specifies data type of each element in the array.

arrayName: specifies name of the variable you are declaring.

size: specifies number of elements allocated for this array.

3.9.4.2 INITIALIZING ONE DIMENSIONAL ARRAYS

- ❖ We have four options for initializing one dimensional arrays.
- ❖ But, first, REMEMBER THIS: Just like regular variables, arrays that have not been initialized may contain "garbage values."
- ❖ But, now, the stakes are even higher. If you declare an array with 31 elements, and forget to initialize it, you end up with 31 garbage values!

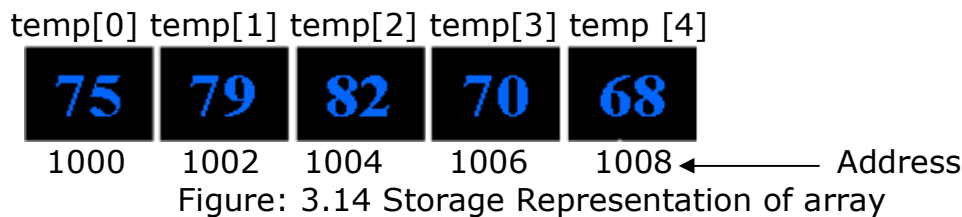
Option 1 Initializing all memory locations

If you know all the data at compile time, you can specify all your data within brackets:

int temp [5] = {75, 79, 82, 70, 68};

during compilation, 5 contiguous memory locations are reserved by the compiler for the variable temp and all these locations are initialized as shown in Fig.3.14.

If the size of integer is 2 bytes, 10 bytes will be allocated for the variable temp.

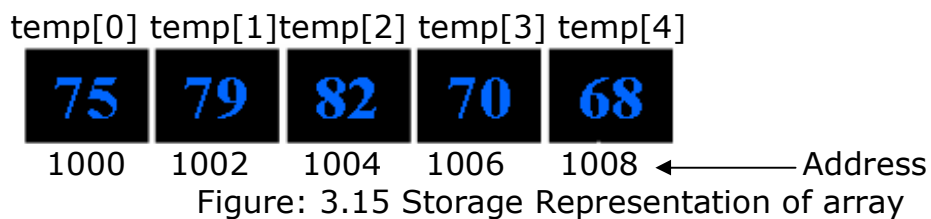


Option 2 initialization without size

If we omit the size of your array, but specify an initial set of data, the compiler will automatically determine the size of your array. This way is referred as initialization without size.

int temp [] = {75, 79, 82, 70, 68};

In this declaration, even though we have not specified exact number of elements to be used in array temp, the array size will be set of the total number of initial values specified. Here, the compiler creates an array of 5 elements. The array temp is initialized as shown in Figure 3.15.



Option 3 Partial Array Initialization

If the number of values to be initialized is less than the size of the array, then the elements are initialized in the order from 0th location. The remaining locations will be initialized to zero automatically.

int temp [5] = {75, 79, 82};

Even though compiler allocates 5 memory locations, using this declaration statement, the compiler initializes first three locations with

75,70 and 82,the next set of memory locations are automatically initialized to 0's by the compiler as shown in figure 3.16

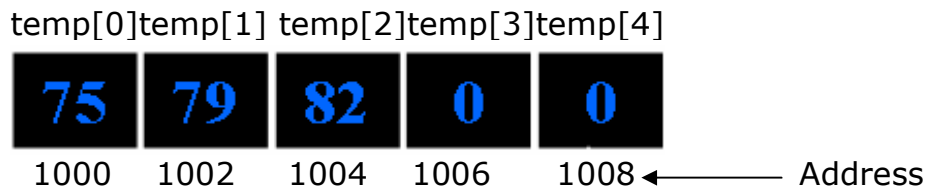


Figure: 3.16 Storage Representation of array

Option 4

If you do not know any data ahead of time, but you want to initialize everything to 0, just use 0 within { }.

For example:

```
int temp [5] = {0};
```

This will initialize every element within the array to 0 as shown in below figure 3.17.

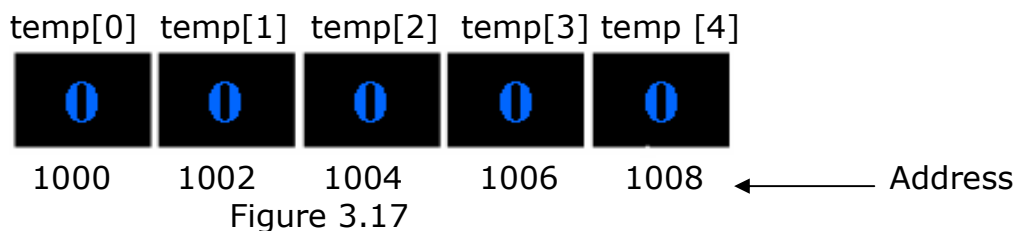


Figure 3.17

If at least one element is assigned then only one element is assigned with the value and all the remaining elements of the array will be zero.

For example:

```
int temp [5] = {5};
```

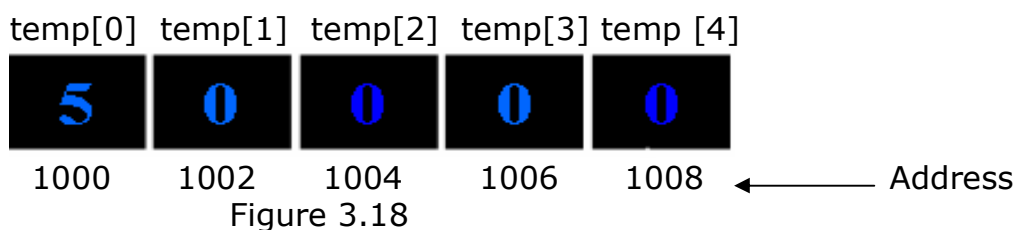


Figure 3.18

The first value is supplied in the first element memory location, remaining all elements are placed with zero.

3.9.4.3 REFERENCING/ACCESSING ONE DIMENSIONAL ARRAY ELEMENTS

- ❖ Suppose we have an array, temperature, for the temperature readings for the year.
- ❖ The subscripts would be 0,1,...,364.
- ❖ To refer to the reading for a given day, we use the name of the array with the subscript in brackets: temperature [4] for the fifth day.

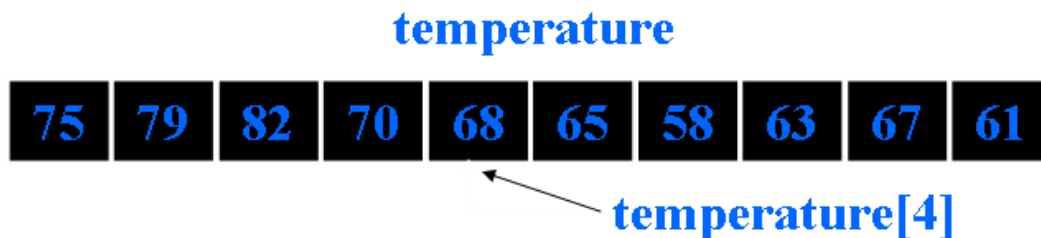


Figure 3.19

- ❖ To assign the value 89 for the 150th day:

temperature [149] = 89;

- ❖ We can loop through the elements of an array by varying the subscript. To set all of the values to 0, say

**for(i=0;i<365;i++)
temperature[i] = 0;**

- ❖ We can not use assignment statement directly with arrays. if a[] , b[] are two arrays then the assignment a=b is not valid .

// C program to read an array elements and find the sum of the elements.

```
#include <stdio.h>
void main()
{
    int a[10];
    int i, SIZE, total=0;
    printf("\n Enter the size of the array : ");
    scanf("%d",&size);
    for (i = 0; i < SIZE ; i++)
        scanf("%d",&a[i]);
    for (i = 0; i < SIZE ; i++)
        total += a[i];
    printf("Total of array element values is %d\n", total);
    getch();
}
```

- ❖ C does not provide array bounds checking.
- ❖ Hence, if we have

```
double stockPrice[5];
printf ("%d", stockPrice[10]);
```

- ❖ This will compile, but you have overstepped the bounds of your array. You may therefore be accessing another variable in your program or another application.

3.9.5. TWO DIMENSIONAL ARRAYS

An array of arrays is called a two-dimensional array and can be regarded as a table with a number of rows and columns:

'J' 'o' 'h' 'n'	is a 3 × 4 array: 3 rows, 4 columns	'J' 'o' 'h' 'n'
'M' 'a' 'r' 'v'		'M' 'a' 'r' 'v'
'I' 'v' 'a' 'n'		'I' 'v' 'a' 'n'

This is an array of size 3 **names [3]** whose elements are arrays of size 4 **[4]** whose elements are characters **char**

3.9.5.1 DECLARATION OF TWO DIMENSIONAL ARRAY

```
elementType arrayName [rows][cols];
```

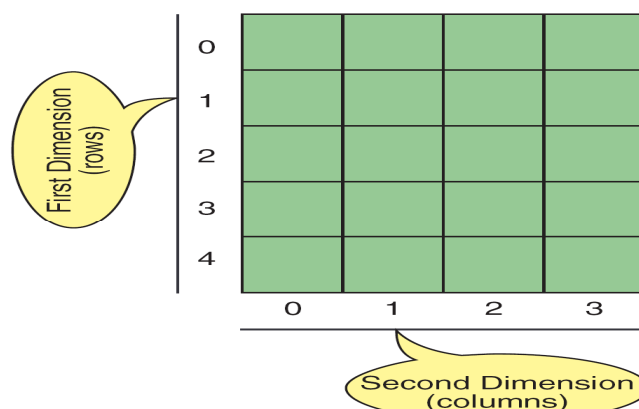


Figure:3. Two-dimensional Array representation

The above figure shows the representation of elements in the two-dimensional array

Example,

```
char names[3][4];
```

in the above declaration

char(elementType) → specifies type of element in each slot

names(arrayName) → specifies name of the array

[3](rows) → specifies number of rows

[4](cols) → specifies number of columns

3.9.5.2 INITIALIZATION OF TWO DIMENSIONAL ARRAYS

- ❖ An array may be initialized at the time of declaration:

```
char names [3][4] = {
                        {'J', 'o', 'h', 'n'},
                        {'M', 'a', 'r', 'y'},
                        {'I', 'v', 'a', 'n'}
                      };
```

- ❖ An integer array may be initialized to all zeros as follows

```
int nums [3][4] = {0};
```

- ❖ In the declaration of two dimensional array the column size should be specified, so that it can arrange the elements in the form of rows and columns.
- ❖ Two-dimensional arrays in C are stored in "row-major format": the array is laid out contiguously, one row at a time:
- ❖ To access an element of a 2D array, you need to specify both the row and the column:

```
nums[0][0] = 16;
printf ("%d", nums[1][2]);
```

//C Program to read and print the elements of the two dimensional array.

```
/* Fill 2-d array, print out values, sum the array. */

#include <stdio.h>
#define M    3 /* Number of rows */
#define N    4 /* Number of columns */
int main(void)
{
    int a [ M ] [ N ], i, j, sum = 0;
    for ( i = 0; i < M; ++i )          /* fill the array */
        for ( j = 0; j < N, ++j )
            scanf ("%d", &a [i] [j]);
    for ( i = 0; i < M; ++i )
    {
        /* print array values */
        for ( j = 0; j < N, ++j )
            printf("a [ %d ] [ %d ] = %d    ", i, j, a [ i ] [ j ]);
        printf("\n");
    }
    for ( i = 0; i < M; ++i ) /* sum the array */
        for ( j = 0; j < N, ++j )
            sum += a [ i ] [ j ];
    printf("\nsum = %d\n\n");
    return 0;
}
```

Produces the output:

```
a[ 0 ] [ 0 ] = 0    a[ 0 ] [ 1 ] = 1    a[ 0 ] [ 2 ] = 2    a[ 0 ] [ 3 ] = 3
a[ 1 ] [ 0 ] = 1    a[ 1 ] [ 1 ] = 2    a[ 1 ] [ 2 ] = 3    a[ 1 ] [ 3 ] = 4
a[ 2 ] [ 0 ] = 0    a[ 2 ] [ 1 ] = 3    a[ 2 ] [ 2 ] = 4    a[ 2 ] [ 3 ] = 5
```

```
sum = 30
```

3.9.6 MULTI DIMENSIONAL ARRAYS

- ❖ C allows three or more dimensions. The exact limit is determined by the compile.
- ❖ The general form of multidimensional array is

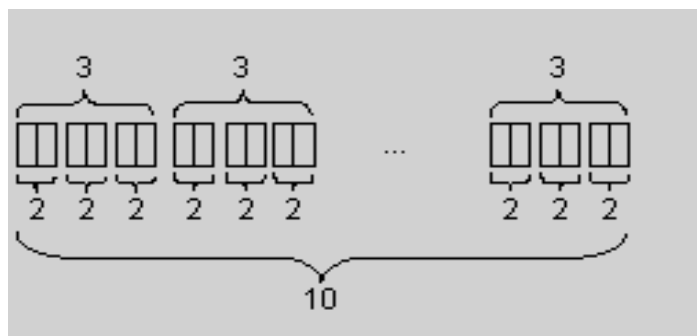
```
type arrayName[s1][s2][s3]....[sm];
```

Where s_i is the size of the i th dimension.

- ❖ Array declarations read right-to-left
- ❖ For example

```
int a[10][3][2];
```

- ❖ it is represented as "an array of ten arrays of three arrays of two ints"
- ❖ In memory the elements are stored as shown in below figure



3.9.7 FUNCTIONS WITH ARRAYS

Like the values of variable, it is also possible to pass values of an array to a function. To pass an array to a called function, it is sufficient to list the name of the array, without any subscripts, and the size of the array as arguments.

for example, the call

```
findMax(a,n);
```

will pass all the elements contained in the array a of size n. The called function expecting this must be appropriately defined.
The findMax function header looks like:

```
int findMax(int x[],int size);
```

The pair of brackets informs the compiler that the argument x is an array of numbers. It is not necessary to specify the size of the array here.

The function prototype takes of the form

```
int findMax (int [], int );
int findMax (int a [], int );
```

```
// C program to find Maximum value in a given
list
#include<stdio.h>
int findMax(int[],int );
void main()
{
    int a[10], n ,i , max;
    printf("\n Enter the size of the array ");
    scanf("%d",&n);
    printf("\n Enter the elements of the array : ");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    max=findMax(a,n);
    printf("\n The Maximum value =%d",max);
    getch();
}
```

```
int findMax(int x[],int size)
{
    int temp;
    temp=x[0];
    for(i=1;i<size; i++)
    {
        if(x[i]>temp)
        {
            temp=x[i];
        }
    }
    return temp;
}
```

Note: The same thing is applied for passing a multidimensional array to a function. Here we need to specify the column size, for example to read two-dimensional array elements using functions it takes of the form

```
int readElements (int [] [5], int ,int );
```

ASSIGNMENT III

1. a) Give some important points while using return statement.
b) Write short notes on the following:
 - i) Actual and formal arguments
 - ii) Global and local variables
2. Define function. What are the different types of function, explain about User defined function and its advantage?
3. What is the scope of variables of type: extern, auto, register and static?
4. a) What is a preprocessor directive?
b) Distinguish between function and preprocessor directive.
c) What is the significance of conditional compilation?
d) How does the undefining of a pre-defined macro done.
5. What is recursion? Write a recursive function power (base, exponent) that when invoked returns base exponent.
6. What are the different standard library functions available in 'C'? Explain with a sample program.
7. a) In what way array is different from an ordinary variable?
b) What conditions must be satisfied by the entire elements of any given array?
c) What are subscripts? How are they written? What restrictions apply to the values that can be assigned to subscripts?
d) What advantage is there in defining an array size in terms of a symbolic constant rather than a fixed integer quantity?
8. Write in detail about one dimensional and multidimensional arrays. Also write about how initial values can be specified for each type of array?